

HTML forms

Forms are used very often when the user needs to provide information to the web server:

- Entering keywords in a search box
- Placing an order
- Subscribing to a mailing list
- Posting a comment
- Filling out a survey
- etc.

An HTML `<form>` element (block-level) contains and organizes **form controls** such as text boxes, check boxes, and buttons that can accept information from web site visitors.

Useful forms require two components:

1. the user interface, that is, the web page containing the form and its controls
2. the server-side processing

Form control elements

- The `<form>` tag contains the form elements on a web page
- The `<input>` tag configures a variety of form controls including text boxes, radio buttons, check boxes, and buttons
- The `<textarea>` tag configures a scrolling text box
- The `<select>` tag configures a select box (drop down list)
- The `<option>` tag configures an option in the select box

Example:

```
<form action="what is this?">  
  Email: <input type="text"  
          name="CustEmail"  
          id="CustEmail" >  
  
  <br>  
  <input type="submit" value="Submit">  
</form>
```

Email:

The <form> tag attributes

- **action**: specifies the server-side program or script that will process your form data; in other words, this attribute specifies *where* to send the form data when the form is submitted
- **method**: specifies *how* to send the form data
 - “get”: (default value) form data passed in URL
 - “post”: form data passed in HTTP Entity Body (a *little bit* more secure)
- **name**: provides a way to reference the form in a script
- **id**: uniquely identifies the form

Now, we'll consider in turn several form elements.

First, we'll look at the <input> element, which accepts input from the user in a variety of ways, depending on the value of its `type` attribute. For example, an input element can be a text box, a checkbox, a password box, a radio button, a button, and more.

Text box: `<input>` element with type “text”

Accepts textual input from the user.

Attributes:

- `type="text"`
- `name`: Only form elements with a “name” attribute will have their values sent to the server when the form is submitted
- `id`
- `size`: specifies the width (in characters) of the text box shown on the screen
- `maxlength`: specifies the maximum length (in characters) of the string that the user is allowed to type in, which could be more or less than the size of the text box
- `value`: defines the initial (default) value of the input box

Example:

```
<form action="form_action.asp" method="get">  
  First name: <input type="text" name="fname"><br>  
  Last name: <input type="text" name="lname"><br>  
  <input type="submit" value="Submit">  
</form>
```

First name:

Last name:

Password box: `<input>` element with type “password”

Accepts textual input that needs to be hidden as it is entered.

Attributes:

- `type="password"`
- `name`
- `id`
- `size`: specifies the width (in characters) of the text box shown on the screen
- `maxlength`: specifies the maximum length (in characters) of the string that the user is allowed to type in, which could be more or less than the size of the text box
- `value`: defines the initial (default) value of the password box

Example:

```
<form action="form_action.asp" method="get">  
  Password:  
  <input type="password" name="pwd" size="20">  
</form>  
<p>  
  Notice that the browser displays asterisks or bullets  
  instead of characters in a password box.  
</p>
```

Password:

Notice that the browser displays asterisks or bullets instead of characters in a password box.

Check box: <input> element with type “checkbox”

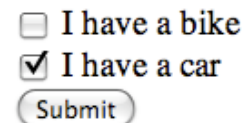
Allows the user to select one or more of a predetermined list of items.

Attributes:

- type="checkbox"
- name
- id
- value: defines the value sent to the server if the box is checked
- checked: specifies that the input element should be preselected when the page loads

Example:

```
<form action="form_action.asp" method="get">  
  <input type="checkbox" name="vehicle1" value="Bike">  
  I have a bike<br>  
  <input type="checkbox" name="vehicle2" value="Car"  
    checked="checked">  
  I have a car <br>  
  <input type="submit" value="Submit">  
</form>
```



☐ I have a bike
☒ I have a car

Click on the submit button, and the input will be sent to a page on the server called "form_action.asp".

<p>Click on the submit button, and the input will be sent to a page on the server called "form_action.asp".</p>

Radio button: `<input>` element with “radio”

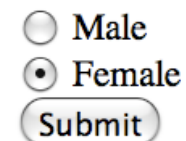
Allows the user to select exactly one from a predetermined list of items.

Attributes:

- `type="radio"`
- `name`: must be the same for all the radio buttons in the group
- `id`
- `value`: defines the value sent to the server if the radio button is selected
- `checked`: specifies that the input element should be preselected when the page loads

Example:

```
<form action="form_action.asp" method="get">  
  <input type="radio" name="sex" value="Male"> Male<br>  
  <input type="radio" name="sex" value="Female"  
    checked="checked">  
    Female<br>  
  <input type="submit" value="Submit">  
</form>
```



Submit/reset buttons: `<input>` element with type “submit/reset”

The submit button sends the form data (that is, the “name=value” pair for each form element) to the web server: when clicked, it triggers the “action” method set in the `<form>` tag.

Attributes:

- `type="submit"`
- `name`
- `id`
- `value`: defines the text on the button

The reset button resets the form fields to their initial values.

Attributes:

- `type="reset"`
- `name`
- `id`
- `value`: defines the text on the button

Button: `<input>` element with type “button”

Creates a generic button without a default action when the button is clicked. Usually a JavaScript function is defined and invoked when the button is clicked.

Attributes:

- `type="button"`
- `name`
- `id`
- `value`: defines the text on the button

Scrollable text area: <textarea> element

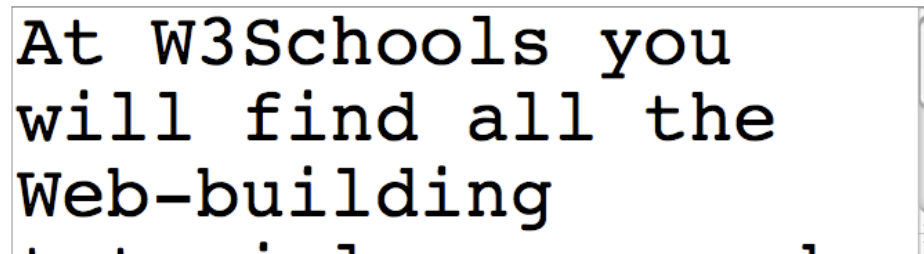
Creates a multi-line text input control in which the user can write an unlimited number of characters.

Attributes:

- name
- id
- cols: specifies the visible width of a text area (better done with CSS)
- rows: specifies the visible height of a text area (better done with CSS)

Example:

```
<textarea rows="3" cols="20"> At W3Schools you will find all the Web-building tutorials you need, from HTML to CSS and JavaScript. </textarea>
```



Drop-down list: `<select>` element

Creates, together with the `<option>` tag, a select list (AKA drop-down list/box, select box, option box)

Attributes:

- `name`
- `id`
- `size`: specifies the number of visible options in the drop-down list
- `multiple`: specifies that multiple options can be selected

The `<option>` element has attributes “selected” and “value”

Example:

```
<form action="form_action.asp" method="get">
  <select name="car">
    <option>Volvo</option>
    <option>Saab</option>
    <option>Mercedes</option>
    <option>Audi</option>
  </select>
  <input type="submit" value="Submit">
</form>
```



Hidden field: `<input>` element with type “hidden”

Defines a hidden field which is not visible by the user but can store a default value or have its value changed via JavaScript code.

Attributes:

- `type="hidden"`
- `name`
- `id`
- `value`: defines the value sent to the server

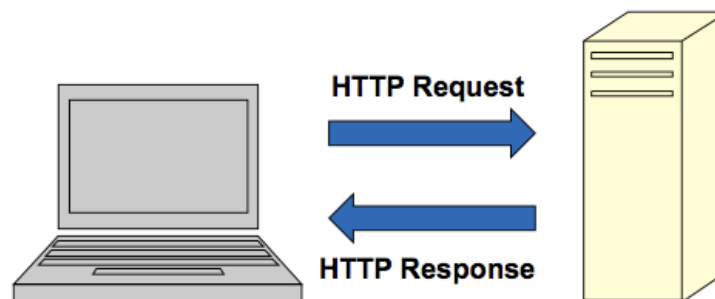
We'll use this type of form control in an upcoming lab.

Server-side processing

Your web browser requests web pages and their related files from a web server.

The web server locates the files and sends them to your web browser.

The web browser then renders the returned files and displays the requested web pages to the user.



Server-side scripting

The **Common Gateway Interface (CGI)** is a standard that defines how web server software can delegate the generation of web pages to another software program called a CGI script that uses data provided by the user (through a form, for example).

One of many technologies in which a server-side script is embedded within a web page document saved with a file extension such as:

- .php (PHP)
- .asp (Microsoft's Active Server Pages)
- .cfm (Adobe's ColdFusion markup language)
- .jsp (Sun's JavaServer Pages)
- .aspx (Microsoft's ASP.Net)

Steps in server-side processing

1. The user invokes server-side processing by submitting form data or clicking a hyperlink on a web page
2. The web server executes a server-side script
3. The server-side script accesses a database, or reads/write files, etc.
4. The web server returns a web page with the requested information or a confirmation of action

Common uses of server-side processing:

- Search a database
- Place an order at an online store
- Send a web page to a friend
- Subscribe to a newsletter

Lab 10 form

Dave's Pizza - Order Form

Phone number:

Pizza size:

Large

Toppings:

☐ Sausage

☐ Onion

☐ Green pepper

☒ **Delivery**

☐ **Carry out**

Reset

Place your order

© 2014

Lab 10 form: Skeleton code

```
<body>
  <form action="http://www.uwosh.edu/cgi-bin/mailto.cgi" method="post">
    <input type="hidden" name="to" value="furcyd@uwosh.edu">
    <input type="hidden" name="from" value="CS125 student">
    <input type="hidden" name="subject" value="Pizza order">
    <table>
      <tr>
        <td colspan="2" class="...">Dave's Pizza - Order Form</td>
      </tr>
      <tr>
        <td>Phone number:</td>
        <td>...</td>
      </tr>
      <tr>
        <td>Pizza size:</td>
        <td>...</td>
      </tr>
      ...
    </table>
  </form>
</body>
```

Lab 10 with borders

Dave's Pizza - Order Form	
Phone number:	
Pizza size:	Large
Toppings:	☐ Sausage ☐ Onion ☐ Green pepper
☒ Delivery ☐ Carry out	
Reset Place your order	
© 2014	